



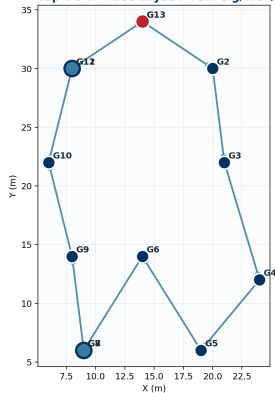
EECS106B DroneRace Project

April 26, 2026

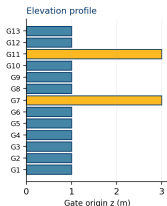
We train an Iris quadrotor to complete a 13-gate race track in Isaac Sim using a PPO pipeline that is deliberately accuracy-first before it becomes speed-focused. wandb.ai/.../dwn4jigo

Task and Track

Top-Down Race Layout from `cfg/task/DroneRaceGeometry`



- 13 YAML gate entries, with Gate 13 duplicating Gate 1 for lap closure
- Elevated gates: G7 and G11 at $z = 3.0$ m
- Stacked reversals: G7→G8 and G11→G12 share XY but flip height / facing
- Hardest maneuvers are the elevated drop-back transitions and lap closure bookkeeping



What makes this course hard

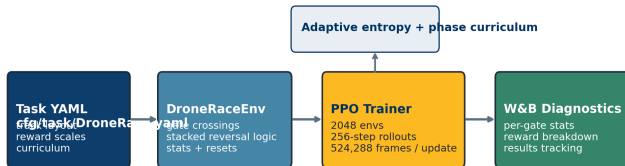
- 13 YAML gate entries, with Gate 13 duplicating Gate 1 only for lap closure.
- Elevated gates 7 and 11 force climb-drop maneuvers instead of flat turns.
- Gates 7→8 and 11→12 share XY coordinates, so the policy must reverse locally.

Interpretation

This is a sequence-legal race, not just waypoint chasing: crossing the wrong gate, or the right gate from the wrong local geometry, should not count.

Training System Overview

Training system grounded in the current codebase

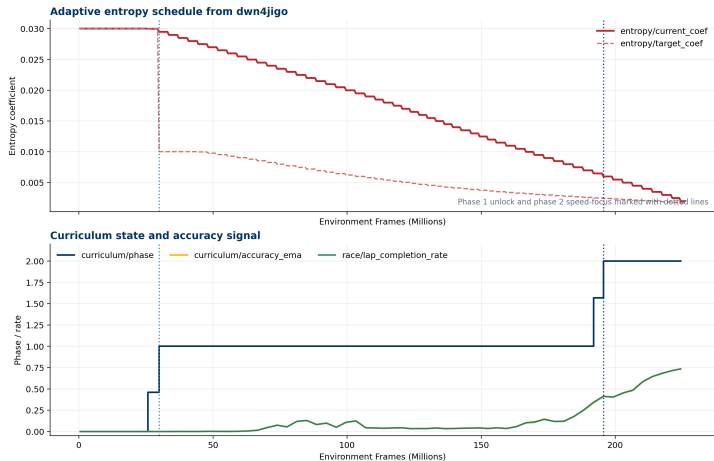


Hydra config chooses the task and PPO overrides; the environment exports curriculum statistics that the trainer folds back into exploration control and W&B plots.

Pipeline facts

- `task=DroneRace` loads the task YAML, which then merges `ppo_cfg: DroneRace.yaml`.
- The active evidence run uses 2048 environments and 256-step rollouts.
- Each PPO update therefore consumes 524,288 environment frames.
- The environment exports curriculum, reset, and per-gate statistics directly into W&B.

Adaptive Entropy Algorithm



Schedule in code

$$\alpha_{\text{target}} = \alpha_{\text{max}} - \frac{\text{accuracy EMA}}{\text{unlock rate}} (\alpha_{\text{max}} - \alpha_{\text{min}})$$

Phase 0 keeps exploration high, phase 1 rebumps entropy when the objective shifts toward speed, and phase 2 decays toward a calmer policy.

Exact run settings

- phase0_coef_max = 0.03
- phase0_coef_min = 0.002
- phase1_rebump_coef = 0.01
- phase1_coef_min = 0.0005
- Current run ended at 0.00197

Other Algorithmic Choices Beyond Vanilla PPO

Why this is not default PPO

- Long-horizon credit assignment with `gamma = 0.998` and `gae_lambda = 0.95`.
- Privileged actor and critic inputs are enabled, so the policy can exploit simulation-only intrinsics during training.
- The critic uses Huber loss, layer normalization, and a wider architecture to stabilize sparse ordered-lap rewards.
- We keep the rollout long enough to cover multiple gate interactions per batch instead of learning from tiny fragments.

Compact objective reminder

$$L^{\text{CLIP}}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

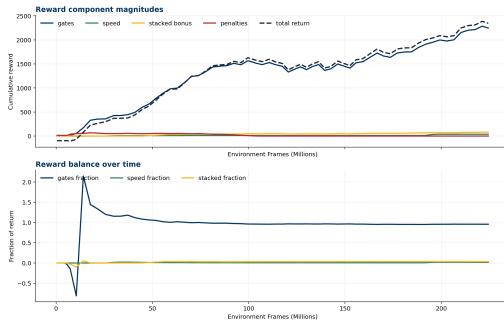
- `clip_param = 0.2`
- `ppo_epochs = 10`
- `num_minibatches = 16`

Practical takeaway

The goal was to make PPO act like a serious racing trainer, not a homework default.

Reward Design

Term	Value	Role
Gate passage	60	Sparse legal crossing reward
Gate sequence	20× streak	Ordered chain bonus
Lap completion	500	Full-course objective
Lap speed bonus	300	Time-aware phase-1/2 bonus
Progress shaping	1.0	Small racing-line guidance
Speed shaping	0.25 / 0.75	Delayed dense speed pressure
Altitude penalty	0.5	Elevated-gate height matching
Centering penalty	0.03	Discourage fly-bys near the plane
Crash penalty	80	Punish divergence or collision

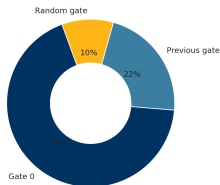


Design principle

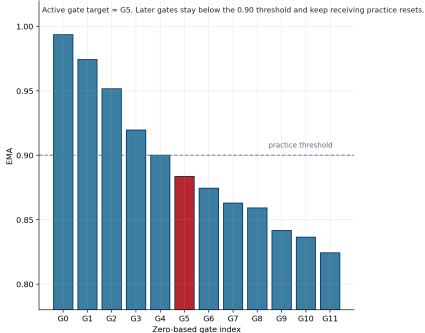
Ordered gate legality must dominate dense shaping, otherwise the policy can learn to fly fast in roughly the right direction without actually racing the course.

Curriculum and Reset Distribution

Reset curriculum mix



Per-gate reset-pass EMA



Three phases

- Phase 0: accuracy-first ordered lap learning.
- Phase 1: bridge phase after the first reliable full-lap signal.
- Phase 2: speed-focus phase after accuracy crosses the target threshold.

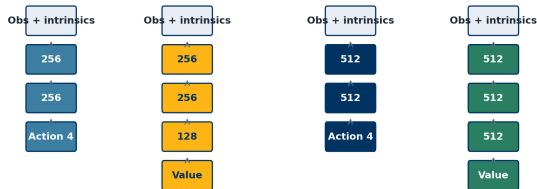
Reset curriculum

The code explicitly allocates resets as 70% gate 0, 20% previous gate, and 10% random gate to keep global course exposure while still drilling the earliest weak bottleneck.

Network Size and Architecture Evolution

Architecture expansion from the older report snapshot to dwn4jigo

Old snapshot widths came from the existing written report. Current widths are pulled from the dwn4jigo run config.



Why the wider model helped:

- More capacity for long-horizon sequence credit assignment
- Better critic fit for sparse ordered-lap rewards
- Enough width to preserve accuracy after speed shaping unlocks

Current widths from dwn4jigo

- Actor: [512, 512]
- Critic: [512, 512, 512]
- Activation: LeakyReLU
- Layer norm: enabled

Why we widened the model

The critic sees a sparse, sequence-heavy objective. The wider value branch improved fit and reduced the risk that the policy would overreact to noisy early returns.

Code-Level Novelty That Helped

Gate-center consistency

`drone_race.py`
`_get_gate_center()` aligns observation, reward, and crossing logic around the aperture center instead of the USD asset origin at the bottom of the gate.

Stacked reversal handling

```
stacked_reversal_grace_steps = 220  
stacked_reversal_dist_threshold = 35
```

These hooks let the drone survive the local target jump around stacked gates long enough to recover instead of crashing immediately.

First-class diagnostics

```
curriculum/reset_gate_XX_pass_ema  
gates/gate_XX_crosses  
entropy/current_coef
```

The trainer logs maneuver-level signals, not just return curves, so we can localize failure modes.

Optional tooling such as auto-stop matters for experiment hygiene, but the main novel pieces that changed learning behavior were the environment-side geometry fixes, reset logic, and richer metrics.

Results Snapshot from the Evidence Run

Important reading note

This run is the main evidence source for the slide deck, but it predates the final gate-11 accounting fix. We therefore use it for performance trends and for bug interpretation.

73.6%

lap completion

13.18 s

lap time

10.11 m/s

max speed

9.86

gates per episode

4.77 m/s

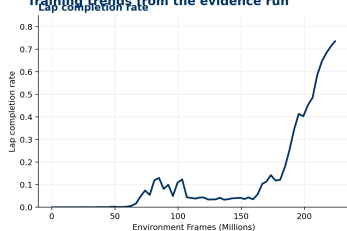
mean speed

22.9%

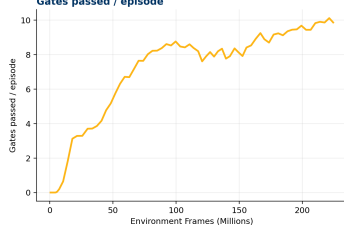
total crash rate

Results Trends and Interpretation

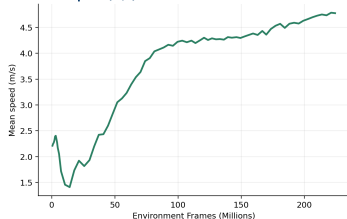
Training trends from the evidence run



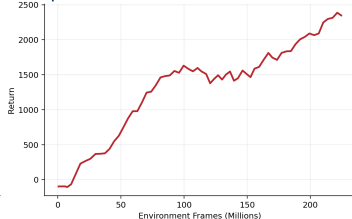
Gates passed / episode



Mean speed (m/s)



Episode return



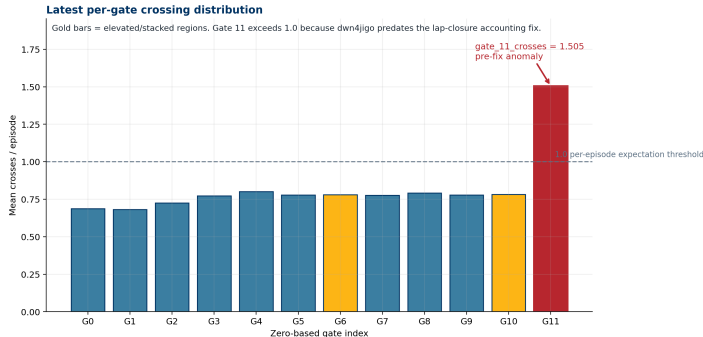
What improved together

- Lap completion and gates per episode rise together, so the policy is learning more than a single lucky maneuver.
- Mean speed climbs later, after the curriculum shifts toward speed focus.
- Return and accuracy stay aligned, which is what we want from an accuracy-first reward hierarchy.

Interpretation

By the end of the run, the policy reaches curriculum phase 2, the accuracy EMA is 0.733, and adaptive entropy has decayed to roughly 0.00197. That is the phase-aware exploration decay we were aiming for.

Per-Gate Analysis and the Gate-11 Bug



Anomalous signal

`gate_11_crosses = 1.505` is not semantically valid as a per-episode gate counter. It should not exceed the expected one-crossing-per-lap bookkeeping.

Root cause and fix

The lap-closure implementation used a duplicated final gate entry, and pre-fix accounting allowed the stacked gate-11 region to be counted incorrectly. Commit 4b808b8 separates the real course gates from the duplicated closure entry so downstream counters ignore the fake lap-closing gate.

Other Bugs and Lessons Learned

Resolved or partially resolved

- **Gate-center mismatch:** earlier observation and reward logic did not target the exact same physical point as the crossing detector.
- **Immediate post-crossing crashes:** grace windows and stacked-reversal thresholds were necessary so successful crossings did not instantly turn into false divergence terminations.

Bug timeline and interpretation guardrail



Observed anomaly: gate_11_crosses = 1.505
This run is the main evidence source, but it predates the bug fix.

Still a caveat

- **Endpoint-only crossing detection:** very fast or awkward local crossings can still be undercounted. A segment-plane intersection test would be more robust.
- **Terminal masking:** PPO GAE currently keys off terminated; once lap completions are common, done/truncated semantics matter more for correct bootstrap behavior.

Main lesson

Better diagnostics changed how we interpreted training. The biggest wins were not only reward changes, but also enough observability to tell whether the model was missing a maneuver, a metric, or a piece of bookkeeping.

Planned MPC / TDMPC Direction

What already exists in the repo

- `cfg/algo/tdmpc.yaml` contains a short-horizon TDMPC configuration with CEM planning.
- `omni_drones/learning/tdmpc.py` already defines the latent dynamics, reward, continuation, and planning code path.
- This is a scaffold, not a reported experimental result for DroneRace.

Why we care

A model-based planner is attractive exactly where PPO struggles most: short-horizon stacked reversals, sharp local geometry changes, and situations where one bad target jump can invalidate a purely reactive policy.

Planned next steps

- Re-run after the gate-11 fix and compare the per-gate distribution.
- Upgrade gate crossing to a segment-based detector.
- Tighten terminal masking around successful lap completion.
- Evaluate whether TDMPC can serve as a teacher, warm-start, or direct planner for the stacked reversal bottlenecks.

Bottom line: PPO is now strong enough to expose where model-based planning might genuinely help, rather than where reward bugs used to dominate the story.